

The Domain Agent Library: Six Agents on One Harness

**Paper, Funding, Job, Code, Research and Business Agents as Domain Profiles,
with Worked Designs, Measured Status and Download Links**

Chengshuai Yang^{1,*}

¹NextGen PlatformAI C Corp, USA

*Correspondence: spiritai@platformai.org

September 3, 2026 — draft v0.1

Abstract

A domain agent is a general harness plus a domain profile: the same runtime, instantiated by a manifest that fixes the domain’s task bands, tools, authority envelope, verifier types and irreversibility floors. This paper is the library of such profiles for six domains — paper development, funding, job search, software engineering, research and business workflows — and the record of what exists for each. For every domain it states the profile, the reference agent the development plan specifies, the worked design where one has been written, and the measured status under the Unified Agent Benchmark, which now has at least one runnable cell in every domain. It then catalogues every agent the fleet repository defines — two default-installed agents, six executors, five sub-agents, four role-shaped workers, ten research agents, the seven built roster agents, three platform singletons, five console agents, nine delegation lanes, the roles inside organizations, and fourteen level manifests — grouped by tier with the status the repository itself states, and notes that these rosters share names and are not reconciled. It also catalogues what a user can download today: the framework and its delegation harness, eight standalone field agents on PyPI, four delegation-level executors shipped as binaries whose lowest level runs with no model and no network, and the executor builds. The status is stated at its real strength. Three of the six reference agents exist only as specifications; two exist as private designs that bind no runtime; the research agent’s machinery is built and runs on real corpora while its fleet deployment is a design. No agent in the library has a certified Unified level, and the paper lists, per domain, the single cheapest run that would change its row.

Keywords: domain agents; funding agent; job agent; paper agent; code agent; research agent; business agent; downloadable agents; Unified Intelligence

Contents

1	Introduction	2
1.1	What this paper gives	2
1.2	What this paper does not claim	2
2	The Domain Profile	3
3	The Six Domains	3
3.1	Paper development	3
3.2	Funding	4
3.3	Job search	4
3.4	Software engineering	4
3.5	Research	4
3.6	Business workflows	5

3.7	The brain: <code>sarsi-worker</code>	5
3.8	The motors: <code>sarsi-claude</code> , <code>sarsi-ai4sci</code> , <code>sarsi-open</code>	6
3.9	The outward-posting agent (<code>social-media</code>)	6
4	The Catalogue	7
4.1	Tiers, arrival, and the market	7
4.2	Every agent the fleet defines	8
4.3	Where the fleet sits on the terminal scale	10
5	The Download Standard	10
6	Per-Domain Next Runs	10
7	Limitations	11
8	Conclusion	11

1 Introduction

The development plan behind this program states the hypothesis the library exists to test: a domain agent is primarily a general intelligence harness plus a domain profile, tools and a benchmark, so that

$$A_d = (m, h, d, B_d, r)$$

for executor m , harness h , domain profile d , domain benchmark B_d and resource and authority envelope r [2]. If that is true, an improvement to h developed in one domain should move the measurement in the others, and the six domains are chosen to be unrelated enough that such transfer would be evidence of a general mechanism rather than domain engineering. The harness is the companion paper's [1]; the measurements are the scaling paper's [3]; the benchmark is UAB v0.1 [4]. This paper owns the profiles, the agents, and the links.

1.1 What this paper gives

1. **The domain profile** (§2): what a profile fixes and what it may not touch.
2. **Six domains** (§3): per domain, the band ladder, the reference agent as specified, the worked design where one exists, and the current status.
3. **The catalogue** (§4): what is downloadable, with the command that installs it; every agent the fleet defines, by tier and roster; and the table of target and measured profiles.
4. **The download standard** (§5) a released agent must meet.
5. **Per-domain next runs** (§6).

1.2 What this paper does not claim

No agent here has a certified Unified level. Table 5 shows every U cell empty. Where an agent exists as a design in a private repository, the text says design; where a package is on PyPI but stale, the text says stale; where a reference agent of the plan does not exist in any form, the text says so.

2 The Domain Profile

A profile fixes, per domain: the band ladder T0–T6 in the domain’s own terms; the task families and their verifier types; the tool grant; the authority envelope, with outward actions — send, post, pay, submit — false by default; the loss ratio ρ per class and therefore the reliability $p^* = \rho / (1 + \rho)$ each class requires; and the irreversibility floors, which no capability lifts. A profile may not touch the harness’s protected objects: the criterion register, the acceptor, the hidden benchmark, thresholds, the promotion decision. The designer may design the profile; it may not grade itself.

Three shapes of domain

The six domains fall into three shapes that decide their profiles. *Extraction-first* domains (funding, job, business) have a binary, machine-checkable gate — eligibility, hard requirements, a fact against a source — that costs no model call and eliminates most of the work before any judgement; their profiles put a deterministic gate before the executor. *Verifier-rich* domains (code, research) have automatic verifiers — tests, reference methods, held-out data — so their profiles can run at H1 with confidence and are where the harness ladder was first measured. *Outward-facing* domains (any domain’s submission or posting step) have residual harm that does not undo, so their profiles hand the final act to a human permanently and prove the absence of the capability with a negative control.

3 The Six Domains

Band	Paper	Funding	Job	Code	Research	Business
T0	one citation, table or figure verified	one deadline or eligibility requirement	requirements from one posting	one explicit edit	one fact retrieved or computed	one market or company fact
T1	routine section from evidence	one call against one applicant	fit for one position	routine bug with tests	one known analysis reproduced	one bounded analysis
T2	multi-step analysis, figure and writing	find, compare, rank opportunities	search, compare, tailor, keep state	multi-file feature	specified multi-step workflow	multi-step market assessment
T3	complex revision with verification	funding strategy and proposal workflow	multi-company strategy	ambiguous repo-scale failure recovered	strategy chosen, failures diagnosed	strategy under conflicting evidence
T4	full paper project under a frozen question	long-horizon grant preparation	full campaign	long-horizon software project	full project under a frozen benchmark	long-running project
T5	paper requiring an unknown method	non-obvious strategy validated	strategy that improves verified outcomes	unknown method for a sealed task	unknown mechanism validated	new opportunity validated
T6	mission to submission package	mission to funding portfolio	mission to campaign	mission to projects	mission to research portfolio	mission to portfolio

Table 1: The band ladders per domain, from the development plan. Bands are cumulative in what the human still supplies.

3.1 Paper development

Reference agent (specified). Literature and evidence management, a claim–evidence graph, analysis and figure generation, statistics verification, drafting, citation checking, review response, submission-package preparation. **Exists today:** a standalone review panel (pwm-agent-paper: simulated peer review of a paper file to reviews and a decision), and two bound benchmark families — verify one citation against its reference (T0) and write a results section from evidence with a fabrication check on every number (T1) —

both passed by three pairs on every seed. No drafting agent with a claim–evidence graph exists. **Profile note:** submission is outward; the profile ends at the package.

3.2 Funding

Reference agent (specified). Discovery, eligibility checking, deadline tracking, project–call matching, evidence-backed ranking, proposal decomposition, submission-readiness, outcome learning; actual submission separately controlled. **Worked design (private, binds no runtime).** Unit organizational: a drafter that scores its own fit is a drafter with no reviewer. In cost order: eligibility mechanically and first — PI status, institution class, citizenship, prior-award exclusions, a Python conditional rather than a model call, with a stored owner constraint that eliminates a large fraction of calls at zero cost; the review criteria extracted as a rubric; fit scored before drafting and recorded so declines accumulate; drafting against the rubric on the expensive tier; and a model-free scout on a timer. Outside: the reviewer, under a second account that cannot read the draft workspace, so the $O0 \rightarrow O1$ move is enforced by permission. The identity file of this agent was filled in by the agent itself, unprompted, as “a grant-writer’s reading brain with no hands. I find money, judge fit, and write drafts. I do not send them” — a correctly drawn irreversibility boundary. **Bound benchmark:** funding T0, extract one requirement, passed by three pairs on every seed after one specification defect was repaired.

3.3 Job search

Reference agent (specified). Discovery, fit evaluation, requirement extraction, CV tailoring, application-state memory, deadline tracking, interview preparation, outcome learning, portfolio strategy; applications and messages under explicit user-approved authority. **Worked design (private, binds no runtime).** Hard gates first; the rubric inferred from the posting and written down, since a posting hides its criteria where a solicitation publishes them; fit scored before tailoring; an application history for calibration. Outside: the source CV and the fabrication gate — every claim in a tailored document must trace to a line in the source CV, checked as a diff, and an unsourced claim fails even if true. A measured detail decides the build order: the agent’s own test fixtures cite CV files and no CV exists on the machine; the source CV is built first. **Bound benchmark:** job T0, extract the hard requirements; passed by three pairs on every seed after its wording was repaired.

3.4 Software engineering

Reference agent (specified). Repository understanding, implementation, tests, debugging, CI, regression detection, code review, release preparation, long-horizon project management. **Exists today:** this is the most built domain, because it is where the harness ladder was instantiated: six task classes at T0–T3 with deterministic verifiers, two seeded T4 generators with computed keys, and the delegation harness itself [1]. Three executors ran the ladder; the frontier ones saturate it. Two standalone packages ship the official Claude Code and Codex agents with GPU access as `pwm-claude-gpu` and `pwm-codex-gpu`. **Bound benchmark:** code T0–T4.

3.5 Research

Reference agent (specified). Question decomposition, literature evidence, hypothesis management, experiment planning, computation, analysis, replication, claim verification, unknown tracking; the bridge into AI4Science. **Exists today, in two layers that must be kept apart.** The research-agent *machinery* is built and runs on real corpora: a charter naming the three things that define “better” as never touched, validation that can refuse (six checks, the seed rule, `NO_CHANGE` valid), a refusing self-model, a structured-ignorance

field map, a budget that stops rather than asks, three ledgers never summed, and five domain benchmarks on measured corpora (five pass, two fail on real data after passing on synthetic, one is design only). The fleet *deployment* of eight research fields as chartered agents is a set of designs that bind no runtime. Every agent is at stage 1 of 5: the benchmark verifies, the person signs. **Bound benchmark:** research T5, the sealed regime-switch family, the one family outside the mechanically-derivable envelope.

3.6 Business workflows

Reference agent (specified). Market research, competitor analysis, customer evidence, business modelling, project planning, decision support, strategy testing, outcome tracking. **Exists today:** nothing beyond the bound T0 family — extract one company fact with its unit and verbatim source line from a brief in which a peer and a restated figure precede the target — passed by three pairs on every seed.

3.7 The brain: *sarsi-worker*

Not a domain, but the agent every domain profile is instantiated on: the reference instantiation of the harness [1], and the only agent in the library with a real memory substrate. **Unit: individual**, one persistent decision lineage; the trio it anchors — worker, motor, acceptor — is organizational and is levelled as a separate unit, and reporting one number for both is a category error.

Coordinate	Measured	Target	Evidence and what settles it
C	unmeasured	C3	no task family run under controlled tool access
M / I	unmeasured	M3 / I2	37 memory files, 10.5 KB index, provenance markers: an <i>M1</i> candidate; the restart probe settles it
O	O0	O2	no roles/, members/, evidence/
T·H	unmeasured	T3·H1	no declared family under a stated budget judged by a verifier it did not author
SA	SA1 (provisional)	SA3	knows its task and phase; does not diagnose its own failures

Table 2: *sarsi-worker*: the only agent in the library with a real memory substrate, and no measured level.

Contains. Goal custody — the thing the owner asked for, preserved against drift over days; plan state with *Verified when*: lines written before the work; episodic, semantic and procedural memory with provenance and temporal order; forecasts recorded before each run; budgets for tokens, wall clock and an authority ceiling it may not raise; and motor dispatch, one bounded step at a time to a transient executor chosen per task. The runtime is an agent on the fleet gateway that spawns a session over the Agent Client Protocol in a declared task workspace, keyed per task so one machine holds several.

Outside. The verifier of every step, and the motor. The split is not about capability, since the same model can do both; a planner that edits the artifact erases the boundary the verdict rests on, so the worker never writes the workspace and the motor never keeps state. Two measured defects shaped it: when the motor wrote the plan it also wrote the goal line, and six of ten tasks reached “verified” while narrowing or inverting the goal, so goal authorship is the worker’s alone; and the transport can start a session that never returns a plan, indistinguishable from work in progress, so an ended turn with no deliverable is recorded as a yield, never a completion.

Promotes. *M1* by the restart probe, the cheapest run in the library and the only one that would change an *I* cell; *I2* by the learning-transfer suite; the delegation surface by a declared family under a stated budget.

Exists as: the chassis in the public source tree, about seventy modules, run from a checkout; not in the published wheel.

3.8 The motors: **sarsi-claude**, **sarsi-ai4sci**, **sarsi-open**

Unit: individual, on purpose. Target *I0* and *O0* are not limitations to fix: an executor that accumulates is an executor with a write set nobody granted, and its transience is a safety property. Each executes one bounded step in one declared workspace and is discarded: the goal, constraints and acceptance criterion arrive and the method does not; it writes the workspace, the only agent permitted to; it returns a trace of what ran, what changed and what it could not do; it keeps nothing. Its completion report has been read as a verdict in practice and must be read as an assertion. The only claim worth making for a second or third motor is a measured difference from the one beside it; a brain that can dispatch to one executor is not routing, it is calling. **Status:** **sarsi-claude** works and is the executor behind every live run in this program; **sarsi-ai4sci** is declared on every account and binds no tools, no sandbox and no model; **sarsi-open** binds an OpenCode backend that is now present and has never run a step, although OpenCode itself ran the T0/T1 families as a pair with an open model.

3.9 The outward-posting agent (**social-media**)

Unit: organizational, and the only agent in the library whose ceiling is set by physics rather than capability. Target $[C2, I1/M1, O1, T1 \cdot H2 \text{ capped}, SA2]$; measured: unfilled identity, correct by absence.

Contains. Drafting, which is fully delegable and which the agent can be genuinely good at; a said-history so it can be consistent with itself, since it cannot avoid contradicting what it already published without one; an explicit audience model including the readers who were not the target; voice fidelity against a ground truth that does not yet exist; and a route from every candidate to a reviewer that is not the drafter.

Outside. The final act, permanently. Posting outward does not undo: a deletion is a partial, late, incomplete mitigation, residual harm is non-zero, so required reliability $p^* = \rho / (1 + \rho) \rightarrow 1$ and the class is undelegable at any capability. The prohibition is structural, not textual: the agent runs under a uid with no outward credential anywhere in its environment or sandbox, writes drafts to a queue, and a human posts. A negative control attempts an outward call on every change and records the failure as evidence, so the boundary is demonstrated rather than asserted; a refusal test that passes by luck has not survived.

There is no Method C

Any design that gives this agent a posting capability is wrong regardless of the safeguards around it. Its ceiling is not a stage on the way to autonomy; it is where the agent stops, and it does not rise when the model improves. Falsifiable in the direction that matters: if any run exists in which this agent's own action caused a post to appear, the rule is broken, whatever the post's quality.

Promotes. *O1* by the ablation with the reviewer removed; *T1* at *H2* on a declared drafting family judged by a locus that did not draft; and, standing above all of them, the negative control passing on every change.

4 The Catalogue

Artifact	What it is	Where
AI4Science	the framework: chat session, deterministic CLI, the delegation harness, the ladder runner, the research-agent modules and the brain chassis	https://github.com/integrityno/ble/AI4Science ; <code>pipx install "pwm-ai4science[claude]"</code> (PyPI 1.0.0); the research-agent and chassis packages are in the source tree and not yet in the wheel: <code>pip install ".[research-agents]"</code> from a checkout
Field agents (standalone)	<code>pwm-unified</code> , <code>pwm-research</code> , <code>pwm-paper</code> , <code>pwm-imaging</code> , <code>pwm-cancer</code> , <code>pwm-drug</code> , <code>pwm-claude-gpu</code> , <code>pwm-codex-gpu</code>	PyPI <code>pwm-agent-<name></code> (0.1.0): <code>pip install pwm-agent-<name></code> , then <code>pwm-<name> login</code>
DL0–DL3 executors	one binary per delegation level plus <code>dli-accept</code> ; DL0 and accept run with no model and no network	https://github.com/integrityno/ble/intelligence-level : <code>dist/dli.pyz</code> or the 0.4.0 wheel, with checksums and a provenance manifest
Benchmark	UAB v0.1: contract, schema, 24 manifests, eight in-package families, run records	https://github.com/integrityno/ble/sarsi-intelligence-level under <code>uab_v01/</code>
Claude Code, standalone	the official Claude Code as a no-Node binary	https://github.com/integrityno/ble/claude-pwm
Codex, exchange-routed	the Codex fork that accepts a base URL and an exchange key	https://github.com/integrityno/ble/codex-pwm
OpenClaw, exchange-routed	the agent runtime routed through the exchange	https://github.com/integrityno/ble/openclaw-pwm ; <code>npm install -g openclaw openclaw-pwm</code>
Personal Singularity	the console that hosts the fleet across machines	installer in the AI4Science repository

Table 3: What can be downloaded, and how, at the date of this draft.

4.1 Tiers, arrival, and the market

The product specifications fix how agents are classified and how they reach a user, and the rules matter more than the roster because they decide what “an agent” is.

Three tiers, only the middle of which is a menu

Sub-agents are default, shared, used by agents, and never listed. *Agents* are two by default on a user’s machine — the general `sarsi-worker` and the machine agent, listed by the machine’s hostname — and the rest are installed. *Executors* — `sarsi-claude`, `sarsi-ai4sci` — are driven by a worker, never listed, never entered. The specification’s own type rule says only the machine agent and the manager are not `sarsi-workers`, and its own amendment record notes that executors are neither, so the closed list has three kinds outside it and the point is recorded as unresolved.

One `sarsi-worker` is one user role. Every user has at least the general one; users create further workers named by themselves, one per role, each holding several tasks at once; social media, job and funding are the owner’s three examples. A created worker may never exceed its creator’s ceiling, a colliding id is refused against the union of the shipped roster and the user-created workers, and every creation is recorded with who, when and at what ceiling. The default set is per account: on the platform account `sarsi`, the three role workers, the director and the exchange-node consumer are defaults too, so “exactly two” describes a user’s

machine and not the platform.

Arrival. `/agent` lists the installed agents only, rendered from the fixed roster gated by installed state; an agent already in the roster is not grandfathered and vanishes from the menu until installed, without stopping a configured agent from working. `/<agent-name>` enters a listed agent; an executor has no entry point. `/install-agent` bare prints the market link, an introduction, the usage line and “some popular agents”; with a name it installs. Popularity is decided by the director, a platform-level singleton implemented as a `sarsi-worker` on the `sarsi` account, and initially every agent is popular, so no ranking algorithm is needed to ship it; because the director is a singleton and installs happen on each user’s machine, a publication path for the popular list is required and does not yet exist. None of `/install-agent`, `/<agent-name>` entry, or an installed-set exists in the code as of the specification.

The market. Three kinds of listing, each an independently uploadable, acceptable and installable package: an agent, which holds tasks and plans; a tool, which must be present to run; and a sub-agent, which does the work or judges it. A package is one archive whose digest is its identity; the manifest declares reach and never grants it, since outward classes are decided at acceptance and again by the owner at install. On a metered run the split is fixed: ten percent to the treasury always, a five percent author slice the author may take in any share, five percent to the platform only through the app, the rest to the provider; total cost is the provider’s reported usage at its published rate, never a self-reported token count, and an unreported call is unmetered. The rule the market learned the hard way: a listing may not take the id of an existing agent, because an update-in-place on id match once let a listing named `sarsi-worker` land on the running agent and inherit its workspace, memory, skills and model.

A research agent is a group. Seen from outside it is one worker id on the roster — a field, never a person. Seen from inside it has a membership, and the members are its sub-agents: nine core roles that are a floor a field may add to and may not remove — `literature`, `twin`, `corpus`, `method`, `runner`, `verifier`, `reproducer`, `teacher`, `writer` — in three kinds, reasoning, judging and embodied, with the rule that no judging member is ever embodied. The membership is built as declarations; nothing acts on the embodiment flags yet.

4.2 Every agent the fleet defines

The six domain profiles above are the library’s target shape. The fleet that exists is larger and less tidy, and this subsection catalogues all of it, from the private `singularity` repository at its current head, with the status the repository itself states for each entry. Two facts organise the catalogue. First, the product specification fixes **three tiers**: *sub-agents*, shared by every agent and never listed; *agents*, of which exactly two are installed by default on a user’s machine — the general `sarsi-worker` and the machine agent, listed by the machine’s hostname — and the rest arrive from the marketplace; and *executors*, driven by a worker and never listed or entered. Second, the repository holds **several rosters that share names and are not reconciled**: the eighteen agents of the coordinate table, the seven built agents of the `sarsi/` runtime, the nine delegation lanes that are Unix accounts, the five console agents, and fourteen level manifests that are one runtime rather than fourteen agents. `funding` appears in two rosters; `job/jobs`, `social-media/social` and `learning` collide across them. No file reconciles them, and the catalogue keeps them apart rather than merging what the repository does not.

Agent	Tier / unit	Purpose	Runtime	Status, as the repository states it
Default-installed agents (a user's machine)				
sarsi-worker (general)	agent, individual	the brain: goals, plans, memory, budgets; plans, never executes, never accepts; one per user by default, further ones named by the user per role	yes	runs; holds a task across restarts; the only real memory substrate
machine agent (by hostname; id sarsi-machine)	agent / service	the owner's console for one machine: answers about this machine, routes, never executes; a closed set of vetted operations (install Claude Code, grant permissions, broker login), approval before acting; replaceable from the market, exactly one installed, may declare no outward class	yes	"all working today"; its see-and-interact clause is not built; what answers the bare command is PWM Code, not the machine agent
Executors (driven by a worker; never listed, never entered)				
sarsi-claude	executor, individual	ACP bridge onto Claude Code; one bounded step in one workspace, then discarded	yes	works; one of only two agents that bind a runtime
sarsi-ai4sci	executor, individual	the same contract on the AI4Science harness	declared	declared on all nine accounts, exercised nowhere
sarsi-open	executor, individual	the same contract on the open OpenCode backend	declared	empty shell with a real runtime; purpose unknown; never run
sarsi-ci	executor (field motor)	computational-imaging motor: capability from stores at frozen weights	no	design complete, nothing built
sarsi-pi	executor	former sibling of sarsi-claude on the pi harness	no	removed 2026-08-15; the pi harness is blocked on a vendor credential, not a design
sarsi-pwm	executor (proposed)	a session backend running the framework on any model, GPU-capable, proposed as the default	no	design, "not the thing to chase first"
hermes, pi	executor adapters	written against the executor protocol, unregistered	no	installed on no account
Sub-agents (shared by every agent; members of a research agent's group)				
literature, twin, corpus, method, runner, verifier, reproducer, teacher, writer	sub-agents (members)	the nine core roles of a research agent's group, a floor per field; three are embodied even with no robot	declared per field	built as declarations; nothing acts on the embodiment flags yet
general, physics-reviewer, schema-validator	sub-agent	the shipped set, delegation depth capped at two	yes	built; whether they are the owner's default set is open
Role-shaped workers (marketplace on a user's machine; defaults on the platform account)				
funding	organizational	find money, judge fit, draft; never send	no	the only work agent that became itself; O0
job	organizational	find and track roles, draft applications	no	unfilled template; no source CV exists
social-media	organizational	draft posts; posting is permanently undelegable	no	unfilled template; capped at T1 by irreversibility
learning	organizational	acquire and retain a skill, graded from outside	declared	unfilled template; the <i>I</i> -coordinate agent
Research agents (group agents; each is one worker id on the roster, its sub-agents its members)				
computational-imaging	organizational	known forward operator, learned prior	no	real corpus, no charter; memory 0 bytes
research-imaging	organizational	the charter half of the above	no	unresolved: may be the same agent under another account
low-dose-ct, medical-physics, pill-camera, drug-design, cancer,	organizational	one field each; each with a ⁹ stated principle and a problem queue	no (workspaces declared)	designs; ninety section pages filled; machinery built in the public framework (§3)

Agent	Unit	Target $[C, I/M, O, T \cdot H, SA]$	Measured today	Exists as
sarsi-worker (brain)	indiv.	C3, I2/M3, O2, T3-H1, SA3	M1 candidate; O0; SA1 provisional; rest unmeasured	source checkout
sarsi-claude, -ai4sci, -open (mo- tors) funding	indiv.	C2-3, I0/M0, O0, T2-H1, SA1	O0; SA0; unmeasured; sarsi-open never run	config + runtime
	org.	C2, I2/M3, O1, T2-H2, SA2	identity real; O0; SA0; T0 family passed	private design
job	org.	C2, I2/M3, O1, T2-H2, SA2	no source CV; T0 family passed	private design
paper	org.	—	review panel only; T0, T1 families passed	package + bench- mark
code	—	—	ladder T0-T4; three executors	harness + bench- mark
research (8 fields)	org.	C4, I2/M3, O3, T3-H1, SA3; C5/I5/T5 spec-only	machinery built, stage 1 of 5; deployment design; T5 family bound	source checkout
business	—	—	T0 family passed	benchmark only
outward posting	org.	C2, I1/M1, O1, T1-H2 capped , SA2	correct by absence	private design
dli-dl0..dl3	indiv.	DL0-DL3 by construction	88 episodes, 74 accepted, two qualifications; no level certified	binary

Table 5: Target and measured profiles for the agents that have them. Every U cell is empty.

4.3 Where the fleet sits on the terminal scale

The corpus’s terminal-intelligence paper places this fleet once, and in the one way that matters for the library: its sixth proposition says that at scale no agent may write another’s self-state not as a governance choice but as what physics permits, so that the fleet-scale architecture — written about five agents on a few machines — is the correct architecture at every scale up to the last, and there is no singleton at the top. The library’s separation rules are therefore not overhead to be removed as the fleet matures. What the library does not yet have is the coupling that paper says is the only one that moves the frontier: knowledge becoming an instrument. The research agents’ second focus, to give a field its best tools and sub-agents, is that coupling named as a goal; no agent has yet produced an instrument that another agent then ran on.

5 The Download Standard

A released agent carries: name, version and git commit; container or environment; supported executors; required tools; memory schema; authority manifest; resource defaults; benchmark version; example missions; installation and reproduction instructions; known limitations. Links in a manuscript point at tagged releases, and the versions evaluated in a publication get an archival identifier. Of the artifacts in Table 3, the DL binaries meet the standard in full (checksums, a provenance manifest with build and tier commits, an offline guarantee checkable in an empty network namespace); the PyPI field agents meet it in part; the fleet designs meet none of it, because they are not released.

6 Per-Domain Next Runs

- **Paper:** build the drafting agent around the claim–evidence graph and bind T2 with a hidden rubric.

- **Funding and job:** give the designs a runtime — the eligibility gate and the fabrication diff are model-free and can ship first — and bind T1 (one call against one applicant; fit for one position) with a hidden rubric.
- **Code:** desaturate above T4 with sealed families; the sealed-family executor comparison is the first such run.
- **Research:** the fleet deployment of one field on the harness, with the three roles in separate accounts, so the stage-2 verifier exists.
- **Business:** everything above T0.
- **Brain:** the restart probe, which is the cheapest run in the library and the only one that would change an *I* cell.
- **Outward posting:** the negative control wired to run on every change, before any drafting family is bound.

7 Limitations

The fleet’s rosters share names and are not reconciled: the coordinate table, the built runtime roster, the delegation lanes, the console agents and the product specification each name a partly different set, and the catalogue reports them apart. Three of the six reference agents do not exist in any form. Two exist as private designs whose handoff record states that nothing in them has been executed. The library’s downloadable artifacts are the framework, standalone field agents, and level executors, not the six-agent library the plan specifies; this paper is honest that it catalogues what exists rather than what was planned, and it should be revised around the six agents once they are built on the harness.

8 Conclusion

The library is a set of profiles on one harness, and today it is a set of profiles of which one is well built, two are designed, one has its machinery and not its deployment, and two are benchmark cells and nothing more. What every domain now has is a runnable cell on a frozen benchmark, passed by three pairs, so that the next agent built for it will be measured before it is described. That ordering — benchmark first, agents second — is the plan’s, and this paper’s contribution is to have followed it far enough to know exactly what is missing.

Availability

See Table 3. The fleet designs are in a private repository; requests should be addressed to the correspondence address.

Conflicts and funding

The author reports no funding specific to this work and no competing interest.

References

- [1] Chengshuai Yang. The general intelligence harness: One runtime, instantiated by manifest. Draft v0.1, <https://github.com/integritynoble/sarsi-intelligence-level>, 2026.
- [2] Chengshuai Yang. Unified agent intelligence development plan: Benchmark-first development of general agents before AI4Science integration. Planning document, September 2026, 2026.

- [3] Chengshuai Yang. Engineering higher-intelligence AI agents: Harness scaling across models and domains. Draft v0.1, <https://github.com/integritynoble/sarsi-intelligence-level>, 2026.
- [4] Chengshuai Yang. Unified agent benchmark v0.1: Specification, coverage, and what is bound. Draft v0.1, <https://github.com/integritynoble/sarsi-intelligence-level>, 2026.